

IFT 212.2

Computer Architecture and Organization

Study Notes — Godswill | UNIPORT, 200 Level Computer Science

Chapter Overview

Computer Architecture and Organization studies how a computer system is built and how its components work together to execute instructions. It is generally split into two related, complementary views:

- Computer Architecture — the "what": the attributes visible to a programmer, e.g. the instruction set, addressing modes, data types, and how the CPU is meant to behave from the outside (its logical/programmer-facing model).
- Computer Organization — the "how": the actual hardware implementation that realizes the architecture, e.g. control signals, buses, memory hierarchy, ALU circuitry, flip-flops, and registers.

This chapter builds up from the smallest working unit to the whole machine: it starts with how numbers and data are represented in binary, moves to the sequential logic building blocks (flip-flops) that give a computer memory, connects those to the combinational logic circuits (encoders/decoders) that route data and instructions, explains how a processor's instruction set is designed (Instruction Set Architecture, including the CISC vs RISC design philosophies), then explains how components physically communicate (the bus system), how devices move data efficiently without tying up the CPU (DMA), and finally how memory is managed and virtualized (the Memory Management Unit).

1. Number Systems and Data Representation

1.1 Why Number Systems Matter

A computer's hardware only understands two electrical states: on and off, represented as 1 and 0. Every piece of data a computer works with — numbers, text, images, instructions — must ultimately be reduced to patterns of binary digits (bits). Number systems give us a formal way to represent quantities using a fixed set of symbols (a "base" or "radix"), and understanding them is the foundation for everything else in this chapter, from how flip-flops store bits to how the ALU performs arithmetic to how addresses are generated on the bus.

1.2 The Common Positional Number Systems

Binary (Base 2): Uses only digits 0 and 1. This is the native language of digital hardware since each bit maps directly onto an electrical high/low or a flip-flop's two states.

Octal (Base 8): Uses digits 0–7. Historically used as a shorthand for binary because each octal digit maps exactly to 3 binary bits.

Decimal (Base 10): Uses digits 0–9. This is the number system humans use naturally, but it is not native to digital hardware.

Hexadecimal (Base 16): Uses digits 0–9 and letters A–F (representing 10–15). Widely used in computing as compact shorthand for binary because each hex digit maps exactly to 4 binary bits (a nibble).

Each system is positional: the value of a digit depends on its position (place value), which is a power of the base. For example, the decimal number present in position i (counting from 0 at the rightmost digit) is multiplied by 10^i . The same logic applies to binary (powers of 2), octal (powers of 8), and hexadecimal (powers of 16).

1.3 Number Base Conversion

Binary to Decimal

Multiply each bit by its corresponding power of 2 (based on position) and sum the results. For example, binary $1011 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = 8 + 0 + 2 + 1 = 11$ in decimal.

Decimal to Binary

Repeatedly divide the decimal number by 2, recording the remainder at each step, until the quotient reaches 0. Reading the remainders from last to first gives the binary equivalent.

Binary to Octal / Hexadecimal

Group binary digits into sets of 3 (for octal) or 4 (for hexadecimal), starting from the rightmost bit, padding with leading zeros if necessary, then convert each group independently.

Octal / Hexadecimal to Binary

Expand each octal digit into its 3-bit binary equivalent, or each hexadecimal digit into its 4-bit binary equivalent, then concatenate.

1.4 Binary Arithmetic

Digital circuits (including the ALU discussed elsewhere in this course, and the flip-flop-based registers covered later in this chapter) perform arithmetic directly on binary values:

- Binary Addition follows simple rules: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=0$ with a carry of 1 — the same "carry" concept as decimal addition, just base 2.
- Binary Subtraction can be done directly, but in practice most digital systems avoid dedicated subtractor circuits and instead perform subtraction by adding the negative of a number (see two's complement below).
- Binary Multiplication and Division follow the same long-multiplication/long-division logic as decimal arithmetic, but using only 0s and 1s.

1.5 Representing Signed Numbers

Since hardware only stores bits, a system is needed to represent negative numbers. Three common schemes:

- Sign-Magnitude: the leftmost (most significant) bit indicates sign (0 = positive, 1 = negative), and the remaining bits store the magnitude. Simple, but has two representations of zero (+0 and -0) and requires special-case logic for arithmetic.
- One's Complement: negative numbers are formed by inverting every bit of the positive value. Still has two representations of zero, and requires an "end-around carry" adjustment during addition.
- Two's Complement: negative numbers are formed by inverting every bit of the positive value and then adding 1. This is the scheme almost universally used in modern computers because it has a single representation of zero and allows addition/subtraction to be done with the same adder circuit, without special-case logic.

This connects directly to the "Sign & Unsigned" distinction noted for processors: signed representation (typically two's complement) allows a range of both negative and positive values within a fixed word length, while unsigned representation dedicates all bits to magnitude, doubling the positive range but disallowing negative values.

1.6 Representing Real (Fractional) Numbers

Whole numbers map naturally onto integer representation, but numbers with a fractional part need a different scheme. The two broad approaches are:

- **Fixed-Point Representation:** a fixed number of bits is reserved for the fractional part (the bits after the "binary point"), and a fixed number for the whole-number part. Simple, but has a limited range and precision.
- **Floating-Point Representation:** the number is stored similarly to scientific notation, split into three fields — a sign bit, an exponent, and a mantissa (significand). The IEEE 754 standard is the most widely used floating-point format, defining exact bit-widths for single precision (32-bit: 1 sign bit, 8 exponent bits, 23 mantissa bits) and double precision (64-bit: 1 sign bit, 11 exponent bits, 52 mantissa bits).

This is why declaring the word length matters, and specifically the number of bits reserved after the decimal/binary point: it directly determines how much range and precision a real number can have, and how many bits a given design needs to allocate.

1.7 Data Representation Summary

Data representation deals with the definite, agreed-upon arrangement of numbers and characters so that hardware can consistently store and interpret them. Key decisions a designer must make include:

- Whether a value is a character or an integer (a character code, such as ASCII or Unicode, is not itself an integer value even though it is stored using integer bit patterns — the interpretation differs).
- Whether a number is a whole number (Integer) or has a fractional part (Integer/Real, i.e. floating point).
- Whether the value needs to represent negative numbers (Signed) or only non-negative numbers (Unsigned).
- How many bits (the word length) are allocated overall, and specifically how many bits are reserved for precision after the decimal point in real-number representations.

2. Instruction Set Architecture (ISA)

2.1 What is an ISA?

The Instruction Set Architecture is the interface between software and hardware — it is the complete set of instructions a processor can execute, along with the rules for how those instructions are formatted, what data types and addressing modes are supported, and how registers and memory are exposed to a programmer or compiler. It is effectively a contract: software written against an ISA will run correctly on any processor that implements that ISA, regardless of the underlying hardware details (the Computer Organization) used to implement it.

2.2 Core Components of an ISA

Instruction Format: How an instruction is encoded in bits — typically split into an opcode (which operation to perform) and one or more operands (the data or addresses the operation acts on).

Addressing Modes: The different ways an instruction can specify where its operand data is located — e.g. directly in the instruction (immediate), in a register (register addressing), or in memory (direct, indirect, indexed addressing, etc.).

Register Set: The number, size, and purpose of the processor's internal storage locations that instructions can operate on directly (built from flip-flops — see Section 3).

Data Types: The kinds of values instructions can operate on directly — integers, floating-point numbers, characters, etc. — tying back into Section 1's number systems.

Instruction Types: The categories of operations available: data movement (load/store/move), arithmetic and logic (add, subtract, AND, OR), control flow (branch, jump, call/return), and input/output operations.

2.3 ISA Design Philosophies: CISC and RISC

When designing an ISA, architects must decide how much work a single instruction should be responsible for, and how uniform or varied those instructions should be. This design decision has historically produced two contrasting philosophies: CISC (Complex Instruction Set Computer) and RISC (Reduced Instruction Set Computer).

CISC — Complex Instruction Set Computer

- Philosophy: pack more functionality into a single instruction, so a programmer (or compiler) needs fewer lines of assembly code to perform a task.
- Instructions can perform multi-step operations (e.g. load from memory, perform arithmetic, and store the result — all within one instruction).
- Variable-length instructions — some instructions are short, others longer, depending on their complexity.
- Many addressing modes, giving flexibility in how operands are specified.
- Instructions often take multiple clock cycles to execute, since they may internally perform several steps.
- Relies heavily on microcode: instructions are broken down internally into smaller micro-operations, which requires more complex control logic to decode and sequence.
- Examples: Intel x86 / x86-64 family.

RISC — Reduced Instruction Set Computer

- Philosophy: keep each instruction simple and fast, and build complex operations by combining several simple instructions instead.
- Fixed-length instructions — every instruction is the same size, making decoding fast and predictable.
- Fewer addressing modes, trading flexibility for simplicity and speed.
- Most instructions execute in a single clock cycle (or close to it), often using a pipelined design where multiple instructions are processed in overlapping stages.
- Load/Store architecture: only specific Load and Store instructions can access memory; all other instructions operate only on registers, which greatly simplifies the hardware.
- Simpler control unit — often hardwired control rather than microcode, since instructions are already simple and uniform.
- Examples: ARM (used in most smartphones), RISC-V, older MIPS and PowerPC processors.

2.4 CISC vs RISC — Comparison

Feature	CISC	RISC
Instruction set	Large, complex	Small, simple
Instruction length	Variable	Fixed
Clock cycles/instruction	Multiple	Mostly 1 (pipelined)

Feature	CISC	RISC
Addressing modes	Many	Few
Memory access	Many instructions can access memory	Only Load/Store instructions
Control unit	Microcode (complex)	Hardwired (simple)
Code size	Smaller (fewer instructions needed)	Larger (more instructions needed)
Hardware complexity	Higher	Lower
Compiler complexity	Simpler compiler, complex hardware	Complex compiler, simpler hardware
Examples	Intel x86/x86-64	ARM, RISC-V, MIPS

2.5 Where CISC and RISC Show Up Elsewhere in This Chapter

- Number Systems and Data Types (Section 1): CISC instructions often support more data types and complex addressing directly in hardware, while RISC keeps operand types simple, pushing more of that flexibility onto software/the compiler.
- Flip-Flops and Control Logic (Section 3): CISC's microcoded control unit is itself a small sequential circuit (built from flip-flops and combinational logic) that steps through micro-operations — meaning a CISC processor generally needs more internal state (more flip-flops) to track its progress while decoding a complex instruction. RISC's hardwired control uses far less sequential logic because each instruction is already close to "micro-op sized".
- Decoders (Section 4): instruction decoding is itself a large-scale decoder circuit — CISC decoders are more complex (they must recognize far more instruction patterns of varying length), while RISC decoders are simpler and faster because instructions are fixed-length and uniform.
- Bus System and DMA (Sections 5–6): RISC's simplicity and pipelining generally demand very efficient, high-throughput memory access (since almost every instruction may need a register or a single memory access), which is part of why RISC designs pair well with fast bus/cache architectures. CISC's complex instructions can do more "work per memory access," which historically mattered more when memory was slow and expensive.

A modern note: the strict CISC vs RISC divide has blurred over time — modern x86 (CISC) processors internally translate complex instructions into RISC-like micro-operations before executing them, borrowing the pipelining and hardwired-style efficiency that made the RISC approach attractive.

3. Flip-Flops (Sequential Logic Basics)

Flip-flops are the basic binary storage elements used in sequential logic circuits. Unlike the purely combinational circuits discussed in Section 4, a flip-flop's output depends not just on its current inputs but also on its previous state — this is what gives digital circuits memory. Each flip-flop can store 1 bit of information, and is typically controlled by a clock signal that determines exactly when the stored value is allowed to change (this is what "edge-triggered" behavior refers to — the flip-flop only updates on a rising or falling clock edge, denoted "↑" in the truth tables below). The primary types covered in this course are SR, JK, D, and T flip-flops.

3.1 SR (Set–Reset) Flip-Flop

Benefit: It is the simplest type of flip-flop, useful for basic latching mechanisms.

Function: It sets the output to 1 when $S = 1$, $R = 0$, and resets it to 0 when $S = 0$, $R = 1$. However, it has an invalid/indeterminate state when both inputs are 1.

Clock	S	R	Q(next)	Description
↑	0	0	Q	No Change (Hold)
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	X	Invalid / Forbidden

The reason $S = 1$, $R = 1$ is forbidden is that both the Set and Reset conditions are being requested simultaneously — the circuit's internal feedback loop cannot settle into a stable, predictable state, and the actual output becomes unreliable (dependent on physical gate delays rather than logical design). This limitation is precisely why the JK flip-flop, described next, was developed.

3.2 JK Flip-Flop

Benefit: It eliminates the invalid state of the SR flip-flop. When both inputs are 1, it toggles (inverts) the current output.

Function: Widely used in counters because of its toggle property.

Clock	J	K	Q(next)	Description
↑	0	0	Q	No Change (Hold)
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	$\bar{Q}$	Toggle (Invert)

The JK flip-flop can be thought of as a refinement of the SR flip-flop: J behaves like S (Set) and K behaves like R (Reset), but the previously forbidden $J=1$, $K=1$ combination is repurposed to mean "toggle" instead of being left undefined. This makes the JK flip-flop strictly more useful, and it can be configured to behave like any of the other flip-flop types (for example, tying J and K together turns it into a T flip-flop).

3.3 D (Data / Delay) Flip-Flop

Benefit: Simplifies input wiring since it only has a single data input. It prevents any ambiguous state entirely.

Function: It captures the value of the D input at the clock edge and delays it to the output by one clock cycle. This makes it the foundation for CPU registers and computer memory (RAM).

Clock	D	Q(next)	Description
↑	0	0	Reset
↑	1	1	Set

Because a D flip-flop simply mirrors its input to its output one clock cycle later, a row of D flip-flops wired to a shared clock forms a register capable of holding a full binary word (e.g. 8, 16, 32, or 64 bits at once). Chaining many such registers together in a grid, with decoders (Section 4) selecting which row to read/write, is the basic structural idea behind RAM.

3.4 T (Toggle) Flip-Flop

Benefit: Highly efficient for frequency division and constructing ripple counters.
Function: If T = 0, it holds the state. If T = 1, it flips (toggles) the output on every clock pulse.

Clock	T	Q(next)	Description
↑	0	Q	No Change
↑	1	$\bar{Q}$	Toggle

Because every toggle halves the output frequency relative to the clock feeding it, chaining T flip-flops together produces a binary counter: the first flip-flop toggles on every clock pulse, the second toggles on every toggle of the first (i.e. half as often), and so on — this is exactly how simple ripple counters and frequency dividers are constructed from basic logic.

4. How Flip-Flops Affect the Building of Coders (Encoders) and Decoders

4.1 Encoders and Decoders Are Purely Combinational Logic Circuits

By definition, standard encoders (which compress multiple inputs into a binary code) and decoders (which take a binary code and activate a specific output line) do not possess memory. They depend entirely on their current inputs, relying on basic logic gates (AND, OR, NOT) rather than flip-flops. Given the same input combination, a combinational circuit will always produce the same output, regardless of what happened before — this is the key difference from the sequential circuits in Section 3.

4.2 How Flip-Flops Intersect with Encoders/Decoders

While flip-flops don't build basic encoders/decoders themselves, they interact extensively with them within a computer's architecture:

1. Memory Addressing (Instruction Decoders)

In a CPU, a register (built from D flip-flops) holds an instruction or a memory address. This address is then passed directly into a decoder to select a specific memory row or activate a specific control unit line.

2. Registers to Priority Encoders

When multiple hardware components request attention (interrupts), their requests are stored in status flags (flip-flops). A Priority Encoder evaluates these flip-flop outputs to determine which requests the CPU should handle first.

3. Sequential Encoders / Decoders

In advanced communication systems, if an encoding scheme depends on past data states (like convolutional codes or state-machine-based line coding), flip-flops are introduced alongside the encoder/decoder circuitry to track the system's current state.

Big picture: encoders/decoders answer "what output corresponds to this input right now?" (combinational, no memory). Flip-flops answer "what happened before, and what state are we in now?" (sequential, has memory). Real CPU subsystems (instruction decoding, interrupt handling, communication coding) combine both — flip-flops hold state, and encoders/decoders act on that state.

5. Bus System

5.1 The Three Bus Types

Address Bus (Unidirectional): Carries the address of data — information about where the data is stored or needs to go in memory.

Data Bus (Bidirectional): Carries data/values about being read or written.

Control Bus: Carries command signals like read, write, and clock signals.

Think of the three buses working together like a library request: the Address Bus says which shelf/book (location), the Data Bus carries the actual book contents back and forth, and the Control Bus carries the instructions like "borrow this" (read) or "return this" (write), plus timing signals (clock) that keep everything synchronized.

5.2 Bus Width

The width of a bus (the number of parallel wires it uses) determines how much information it can carry at once. A wider Address Bus can address more distinct memory locations (an n -bit address bus can address 2^n locations), while a wider Data Bus can transfer more bits of data in a single operation, directly affecting performance.

5.3 Synchronous vs Asynchronous Buses

- Synchronous Bus: all data transfers are coordinated by a shared clock signal, with events happening at fixed, predictable intervals. Simpler to design, but every connected device must operate within the same clock timing constraints.
- Asynchronous Bus: transfers use a handshaking protocol (e.g. a request signal followed by an acknowledge signal) instead of a shared clock, allowing devices operating at different speeds to communicate without being locked to a single timing standard.

5.4 Bus Arbitration

Since multiple devices may need to use a shared bus, but only one device can control it at any given moment, a bus arbitration scheme is needed to decide who gets access and when. This is directly relevant to DMA (Section 6), where a device must request and be granted control of the system bus before it can transfer data directly to memory.

6. Direct Memory Access (DMA)

6.1 What DMA Is

A method that allows an I/O device to transfer data straight to the main memory, by taking over the system bus — without requiring the CPU to manage every individual data transfer step.

6.2 DMA Controller

A special chip/hardware module that manages the transfer on behalf of the I/O device.

Why it matters: It frees up the CPU from doing repetitive, boring data-transfer tasks, letting it continue executing other instructions while the transfer happens in the background.

6.3 Steps DMA Uses the Bus System

- Bus Request — the I/O device (via the DMA controller) signals that it wants to release/acquire the system bus for a transfer.
- Bus Grant — the CPU grants the request, temporarily relinquishing bus control.
- Interrupt Signal — once the transfer is complete, an interrupt signals the CPU that execution/attention can resume.

6.4 Modes of DMA Transfer

- Burst Mode — the DMA controller takes control of the bus and transfers an entire block of data in one go before releasing the bus. Fastest for bulk transfer, but it locks the CPU out of the bus for that whole burst.
- Cycle Stealing Mode — the DMA controller transfers one word at a time, then releases the bus back to the CPU between transfers. Slower than burst mode, but the CPU isn't blocked for long stretches.
- Transparent Mode — the DMA controller only uses the bus during cycles when the CPU isn't using it, so there's no interruption to CPU operation, but it's the slowest since it must "wait its turn" and keeps the bus until the system task is completed.

6.5 Advantages and Disadvantages

The DMA and System Bus work together to let hardware move data from computer memory without using the CPU. This significantly improves overall system throughput for data-heavy operations (like disk transfers or network traffic), though it does add hardware complexity (the DMA controller itself) and requires careful arbitration so DMA transfers don't starve the CPU of bus access for too long.

7. Memory Management Unit (MMU)

7.1 Virtual Memory

Creates the illusion of a larger main memory space by combining physical RAM and secondary (disk) storage. This allows programs to be written and run as though a large, contiguous block of memory is available, even if physical RAM is smaller than what all running programs collectively demand.

7.2 System Bus

A physical group of wires or connections that the MMU (along with the CPU and memory) relies on to move addresses and data during translation and access.

7.3 Core Concepts and Terminology

Virtual Address Space: The set of logical addresses generated by the CPU.

Physical Address Space: The set of actual addresses in physical RAM.

Pages: Fixed-size blocks of virtual memory.

Frames: Fixed-size blocks of physical memory — pages are mapped into frames.

7.4 How Address Translation Works

The CPU works with virtual addresses (organized into Pages) as if it has a large, private block of memory. The MMU translates these into physical addresses (organized into Frames) in actual RAM, typically using a Page Table that records which virtual page maps to which physical frame. To speed this lookup up, processors typically use a Translation Lookaside Buffer (TLB) — a small, very fast cache of recent page-to-frame translations, avoiding the need to consult the full page table on every single memory access.

7.5 Page Faults and Swapping

When RAM is full, or a program tries to access a page that currently isn't loaded into physical memory, a page fault occurs. The operating system responds by moving a less-used page out to secondary storage (disk) — often called swapping — to free a frame, then loads the requested page into that frame. This swapping between RAM and disk is exactly what creates the "illusion" of more memory than physically exists, though excessive swapping (thrashing) can significantly slow a system down since disk access is far slower than RAM access.

7.6 Segmentation (Related Concept)

An alternative (or complementary) approach to paging is segmentation, where memory is divided into variable-sized, logically meaningful blocks (segments) — such as a code segment, data segment, and stack segment — rather than fixed-size pages. Some systems combine both approaches (segmented paging) to get the logical organization benefits of segmentation with the simpler allocation benefits of fixed-size paging.

Chapter Summary

Putting the whole chapter together:

- Number Systems and Data Representation define how numbers, characters, and real values are encoded in binary — the foundation before any hardware can act on data.
- Instruction Set Architecture (ISA) defines the contract between software and hardware — what instructions exist, how they're formatted, and how operands and addressing work. CISC and RISC are two contrasting philosophies for designing that instruction set, trading off instruction complexity against hardware simplicity and execution speed.
- Flip-Flops are the sequential building blocks that give digital circuits memory (SR, JK, D, T — each with different behavior and use case), and they underpin registers, counters, and RAM.
- Encoders/Decoders are combinational (memoryless) circuits, but they work hand-in-hand with flip-flop-based registers and state machines throughout a real CPU (instruction decoding, interrupt priority, communication coding).
- The Bus System (Address, Data, Control) is the shared communication highway that lets the CPU, memory, and I/O devices exchange addresses, data, and commands, using arbitration and either synchronous or asynchronous timing.
- DMA lets I/O devices use the bus system directly to move data to/from memory without tying up the CPU, using Burst, Cycle Stealing, or Transparent modes.
- The Memory Management Unit uses Pages and Frames (with Page Tables and a TLB) to translate virtual addresses to physical addresses, creating Virtual Memory — the illusion of more memory than physically exists, at the cost of occasional page faults and swapping.